

```

#include <iostream>
#include <vector>
#include <algorithm>

class TrainSolver {
public:
    TrainSolver(int numCases) : t(numCases) {} // Constructor that initializes the number of cases

    void readData() { // Method to read input data for all test cases
        for (int i = 0; i < t; ++i) { // Loop over each test case
            int numTrains;
            std::cin >> numTrains; // Read the number of trains for the current test case
            std::vector<int> trains(numTrains); // Create a vector to store train data
            for (int j = 0; j < numTrains; ++j) {
                std::cin >> trains[j]; // Read each train's arrival time
            }
            trainData.push_back(trains); // Store the train data for the current test case
        }
    }

    void solveAll() { // Method to solve all test cases
        for (const auto& trains : trainData) { // Iterate over each set of train data
            solve(trains); // Solve for the current set of train data
        }
    }

private:
    int t; // Number of test cases
    std::vector<std::vector<int>> trainData; // Container to store all train data for all test cases

    void solve(const std::vector<int>& trains) { // Method to solve a single test case
        int n = trains.size(); // Get the number of trains
        std::vector<int> dp1(n, 1), dp2(n, 1); // Initialize dynamic programming arrays
        int ans = 0; // Initialize the answer variable

        for (int i = n - 1; i >= 0; --i) { // Iterate from the last train to the first
            for (int j = n - 1; j > i; --j) { // Iterate over all trains after the i-th train
                if (trains[i] > trains[j])
                    dp1[i] = std::max(dp1[i], dp1[j] + 1); // Update dp1 if the current train is greater
                else
                    dp2[i] = std::max(dp2[i], dp2[j] + 1); // Update dp2 if the current train is not greater
            }
            ans = std::max(ans, dp1[i] + dp2[i] - 1); // Update the answer with the maximum length
        }

        std::cout << ans << std::endl; // Print the answer for the current test case
    }
};

int main() { // Main function
    int t;
    std::cin >> t;
    TrainSolver solver(t); // Create an instance of TrainSolver with t test cases
    solver.readData(); // Read input data for all test cases
    solver.solveAll(); // Solve all test cases
    return 0; // Return 0 to indicate successful execution
}

```